
Table of Contents

.....	1
Part 1	1
Part 2	2
Functions	5

```
clc; clearvars -except P psi; close all
```

```
load energy_data.mat
addpath("./mcmcstat/")
```

Part 1

```
%{
Setting up a particle swarm optimization to guess the coefficients the
model should use
%}
```

```
numVars = 4; % order of the model
```

```
% Bounds that the particles can guess between
LB = -20 * ones(numVars,1);
UB = 20* ones(numVars, 1);
```

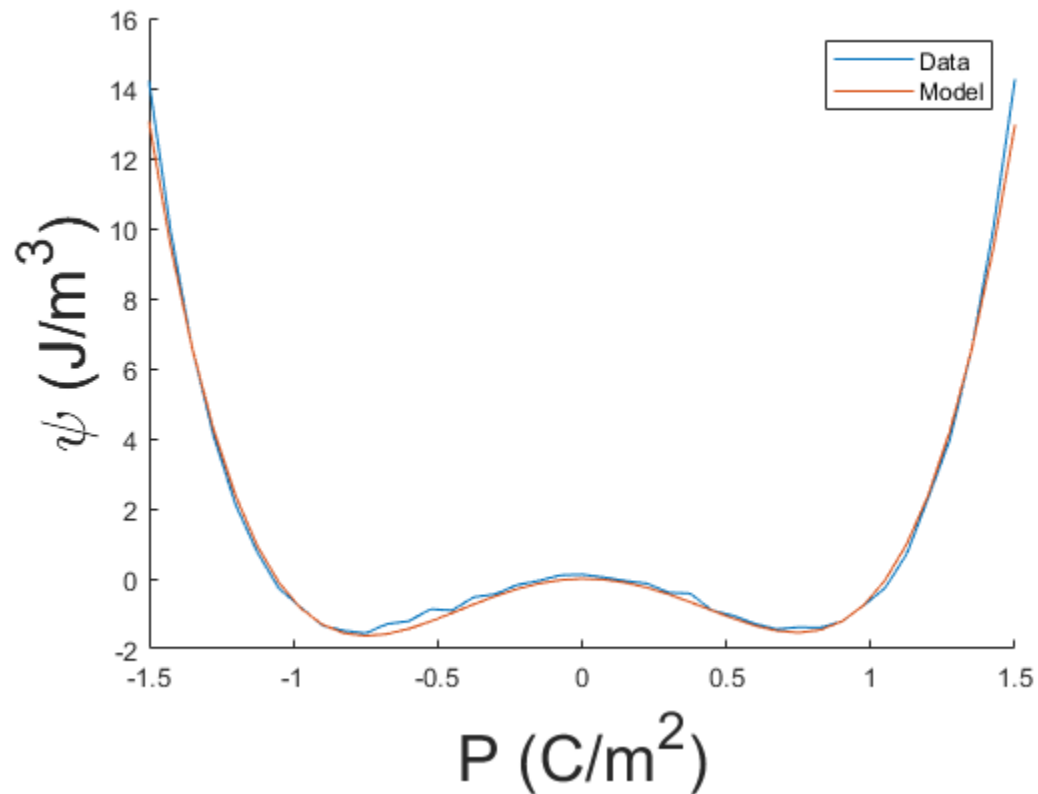
```
% Cost function used by the particle swarm
data.xdata = P;
data.ydata = psi;
data.xdata = data.xdata(:);
data.ydata = data.ydata(:);
```

```
costFunctionHandle = @(theta, numVars) CostFunction(theta,data);
```

```
% Run particle swarm
optimalTheta = particleswarm(costFunctionHandle, numVars, LB, UB);
```

```
% Create y vals based on the optimized coefficients
modelPsi = myPolynomialFunc(optimalTheta, data);
```

```
% Plot
figure()
hold on
plot(P, psi, 'DisplayName', 'Data')
plot(P, modelPsi, 'DisplayName', 'Model')
xlabel('P (C/m^2)', 'FontSize', 24);
ylabel('\psi (J/m^3)', 'FontSize', 24);
legend()
saveas(gcf, "PSO_Guess.jpg")
```



Part 2

```
ssfun = @CostFunction;
model.ssfun=ssfun;

% Using optimized vals as initial guesses
params = {
    {'a1', optimalTheta(1), -10, inf}
    {'a2', optimalTheta(2), -10, inf}
    {'a3', optimalTheta(3), -10, inf}
    {'a4', optimalTheta(4), -10, inf}
};

% Chains

model.sigma2 = 1e-4;          %initial guess on variance
model.S20 = model.sigma2;    %prior for sigma2
model.N0 = 1;                %prior accuracy for S20
options.updatesigma = 1;     %update variance as part of the inference
options.method = 'dram';     %this applies the DRAM algorithm
model.N = length(data.xdata); %number of data points

options.nsimu = 10000; %number of iterations in the Metropolis method
[results, chain, s2chain]= mcmcrun(model,data,params,options);
chainstats(chain,results) %print chain statistics
```

```

figure(2)
mcmcplot(chain(1000:end,:),[],results,'denspanel',2);
saveas(gcf, "Density.jpg")

figure(3); clf
mcmcplot(chain(1000:end,:),[],results.names,'chainpanel')
xlabel('Iterations','FontSize',24)
ylabel('Parameter value','FontSize',24)
saveas(gcf, "Chains.jpg")

figure(4)
mcmcplot(chain,[],results,'pairs');
saveas(gcf, "Pairs.jpg")

```

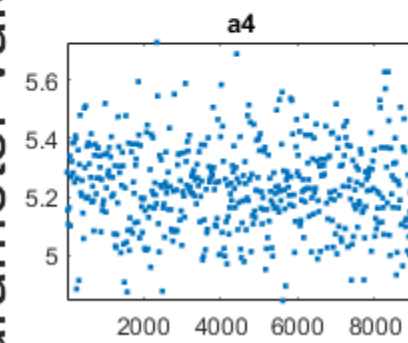
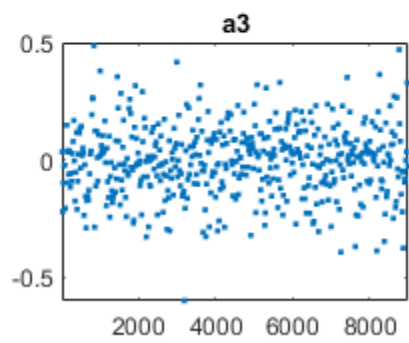
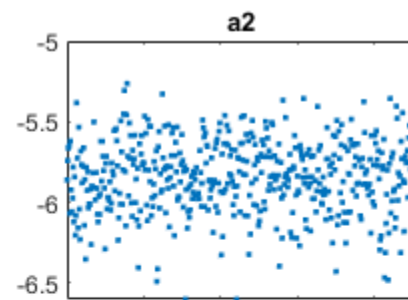
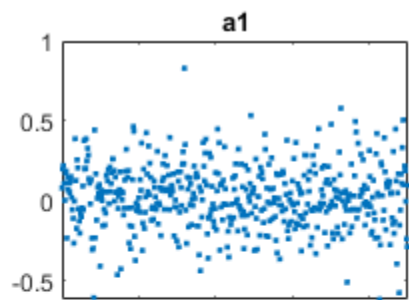
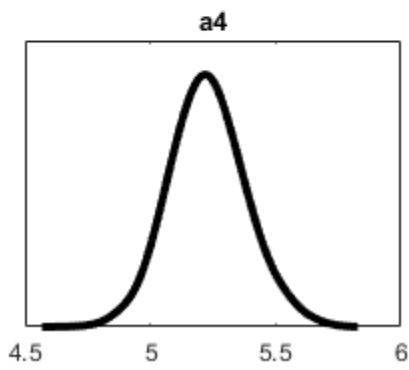
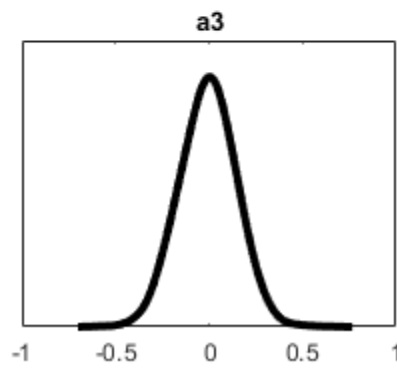
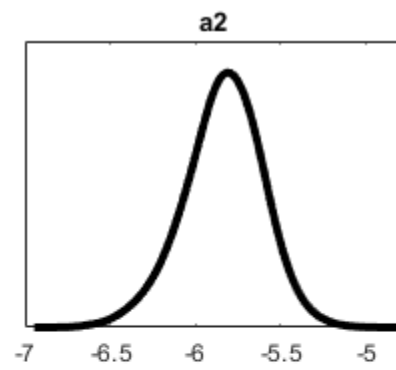
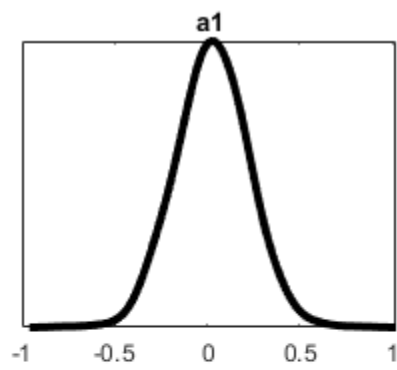
Sampling these parameters:

```

name    start [min,max] N(mu,s^2)
a1: 0.0933889 [-10,Inf] N(0,Inf)
a2: -5.68417 [-10,Inf] N(0,Inf)
a3: -0.0567484 [-10,Inf] N(0,Inf)
a4: 5.09397 [-10,Inf] N(0,Inf)
MCMC statistics, nsimu = 10000

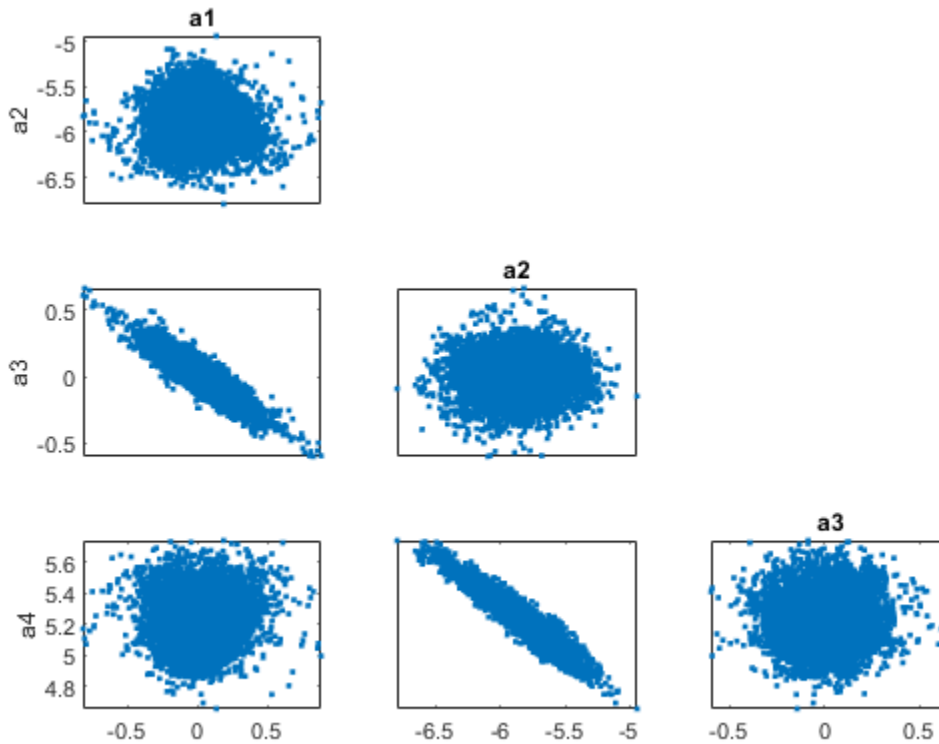
```

	mean	std	MC_err	tau	geweke
a1	0.029754	0.18743	0.0068261	17.154	0.60137
a2	-5.8372	0.22285	0.0063909	13.742	0.99947
a3	-0.0038718	0.14381	0.0047169	15.523	0.78357
a4	5.23	0.14684	0.0040687	12.639	0.99858



Parameter value

Iterations



Compute the credible and prediction intervals

```

modelfun1 = @(d,th)myPolynomialFunc(th,d); % NOTE: the order in which d and
th appear is important

nsample = 500; %number of sample iterations of the model used to construct
the interval bounds
    %the default interval bounds are 95% prediction/credible
    %bounds
out = mcmcpred(results,chain,s2chain,data.xdata,modelfun1,nsample);
figure(5)
modelout = mcmcpredplot(out);
hold on
plot(data.xdata,data.ydata,'b.--','linewidth',1)
hold off
xlabel('P (C/m^2)','FontSize',24);
ylabel('\psi (J/m^3)','FontSize',24);
legend('95% Prediction Interval','95% Credible Interval','Model
Fit','Simulated Data','Location','Best')
saveas(gcf, "Confidence.jpg")

```

Functions

```

% This function takes in x values of interest and polynomial coefficients
% and spits out corresponding y values
function yVals = myPolynomialFunc(coeff, data)

```

```
    if class(data) == "struct"
        xVals = data.xdata;
    else
        xVals = data;
    end
    numParams = numel(coeff);

    for i = 1:numParams
        xMat(:, i) = xVals.^i;
    end

    yVals = xMat * coeff(:);
end

% Taking the difference between model data and experimental data as the
% cost
function cost = CostFunction(theta, data)

    simulatedY = myPolynomialFunc(theta, data.xdata);

    cost = sum(abs(data.ydata - simulatedY));

end

Optimization ended: relative change in the objective value
over the last OPTIONS.MaxStallIterations iterations is less than
OPTIONS.FunctionTolerance.
```

Published with MATLAB® R2024b